Informe de validación y pruebas

Despliegue académico de modelo Random Forest | Integración, reproducibilidad y límites de uso

1. OBJETIVO Y JUSTIFICACIÓN DEL ENTORNO DE EJECUCIÓN

El objetivo de esta fase fue verificar que la aplicación desarrollada con Docker ejecutara predicciones consistentes con los resultados de la Actividad 6, asegurando la correcta integración de sus componentes y la validación de entradas.

El desarrollo se realizó utilizando un equipo corporativo autorizado por el banco, sujeto a controles de ciberseguridad que restringen la instalación y ejecución local de determinados componentes, incluido Docker. Ante esta limitación, se optó por utilizar GitHub Codespaces, un entorno remoto accesible desde el navegador que permitió ejecutar la aplicación y realizar las pruebas sin modificar las restricciones de seguridad del dispositivo.

Esta alternativa permitió obtener y verificar las evidencias técnicas previstas para la actividad. Por ello, se solicita considerar su validez conforme a los objetivos y criterios de evaluación de la rúbrica, tomando en cuenta que el despliegue se ejecutó en un entorno remoto y no mediante una instalación local.

Elemento	Configuración verificada
Dataset	South German Credit, 1,000 registros; 20 variables de entrada.
Partición	800 registros de desarrollo y 200 de prueba reservada.
Algoritmo	Random Forest, 160 árboles, min_samples_leaf=3, random_state=42.
Umbral	0.28 para clasificar riesgo alto.
Tecnologías	Python 3.11, scikit-learn 1.6.1, FastAPI, Streamlit, Docker Compose.

2. RESULTADOS DEL MODELO REPRODUCIDO

El modelo se reconstruyó utilizando la misma configuración de la Actividad 6 y sus resultados se compararon con los registros originales. La diferencia máxima entre las probabilidades fue de apenas 9.71×10^{-17}, lo que confirmó su reproducción. Posteriormente, se verificó que la API generara las mismas 200 clasificaciones del conjunto de prueba, con probabilidades consistentes con las referencias, considerando el redondeo de los resultados.

Indicador	Resultado	Interpretación
ROC-AUC	0.7588	Discriminación moderada en prueba reservada.
Exactitud	63.0%	126 clasificaciones correctas de 200.
Sensibilidad: riesgo alto	78.3%	47 de 60 casos de riesgo alto detectados.
Especificidad	56.4%	79 de 140 casos no altos identificados.
Precisión: riesgo alto	43.5%	47 aciertos entre 108 alertas de riesgo alto.

2.1. Matriz de confusión — prueba independiente

Real / Predicción	No alto	Alto
No alto (n=140)	TN = 79	FP = 61
Alto (n=60)	FN = 13	TP = 47

Los resultados muestran que el modelo logró identificar el 78.3% de los casos de riesgo alto, pero su especificidad fue de 56.4%, inferior al mínimo de 60% establecido en la Actividad 6. Esto significa que 61 de los 140 casos que no correspondían a riesgo alto fueron clasificados incorrectamente en esa categoría. En consecuencia, aunque el modelo presenta capacidad para identificar operaciones de mayor riesgo, no cumplió con todos los criterios de aceptación definidos previamente, por lo que se mantiene el dictamen NO-GO para uso bancario real.

3. PRUEBAS FUNCIONALES Y DE INTEGRACIÓN

Las pruebas realizadas permitieron confirmar tanto el funcionamiento individual de los componentes como su correcta integración. Se verificó que la interfaz enviara la información a la API, que esta ejecutara el modelo y que los resultados se mostraran correctamente al usuario. Asimismo, la comparación de 200 predicciones con los registros de referencia confirmó que el despliegue conserva el comportamiento del modelo validado en la Actividad 6.

Caso	Comportamiento observado	Estado
GET /health	HTTP 200; estado operativo y 20 variables.	PASS
GET /schema	20 variables y umbral 0.28.	PASS
POST /predict	Respuesta JSON con probabilidad y clase.	PASS
200 solicitudes históricas	Probabilidades y clases consistentes con referencia.	PASS
Frontend → API	Streamlit mostró resultado y JSON coherentes.	PASS
Docker Compose	Servicios API y frontend: healthy.	PASS

4. CASOS EXTREMOS Y CONTROL DE ENTRADA

Entrada anómala	Respuesta observada
Variable requerida ausente	HTTP 422.
Variable adicional desconocida	HTTP 422.
Importe negativo	HTTP 422.
Texto donde se espera entero	HTTP 422.
Categoría no incluida en el dominio	HTTP 422 después de corrección v2.

Hallazgo y corrección: la primera versión aceptaba una categoría inexistente. Se incorporó validación de dominio desde form_schema.json en FastAPI y se reforzó el test para fallar automáticamente si el servidor deja de rechazarla. Tras reconstruir y reiniciar el contenedor, las 200 predicciones siguieron coincidiendo con las originales.

Alcance de estas pruebas: los rangos numéricos se configuraron con mínimos y máximos observados del dataset y no constituyen política de originación. No se documentan pruebas de carga, seguridad ofensiva, disponibilidad prolongada ni ejecución en producción.

5. EVIDENCIAS TÉCNICAS DISPONIBLES

Archivo / evidencia	Qué acredita
evidence/pruebas_preliminares.txt	Detección original de categoría fuera de dominio.
evidence/pruebas_validacion_v2.txt	Corrección y repetición de tests de entrada.
evidence/pruebas_compose_final.txt	200 predicciones y pruebas exitosas con Compose.
artifacts/metadata.json	Variables, umbral y matriz de confusión histórica.
docker compose ps (captura)	Dos servicios healthy en Codespaces.
Streamlit (captura)	Formulario, inferencia y respuesta JSON visualizada.
evidence/imagenes/01–06 (PNG)	Capturas de modelo, integración, despliegue, corrección de entradas y repositorio público. Ver INDICE_EVIDENCIAS.txt.

6. EVALUACIÓN DE RIESGOS Y CONCLUSIONES

Se corroboró un prototipo funcional: el modelo es reproducible, la API y la interfaz intercambian información vía HTTP, Docker Compose gestiona ambos componentes y los controles básicos de entrada rechazan los casos adversos previstos. La validación conserva trazabilidad con las Actividades 5 y 6.

La aprobación de las pruebas de software no significa que el modelo sea apto para decisiones reales de crédito. Persisten limitaciones de calidad predictiva, representatividad del dataset, calibración, supervisión humana y gobierno del modelo. Recomendación: conservar el uso exclusivamente académico; cualquier eventual despliegue institucional requeriría una nueva validación y autorización formal.

Considero que esta fase demostró la diferencia entre software funcional y modelo apto para decisiones bancarias. Ante las restricciones del equipo, prioricé un entorno reproducible y sin pagos, así como verificar resultados antes que añadir herramientas. La corrección del control de entradas confirmó la importancia de probar casos adversos. Mantengo el dictamen NO-GO para cualquier uso crediticio real.

7. REPRODUCIBILIDAD Y REFERENCIAS

Repositorio: https://github.com/jalfredolarrea-debug/Fase2-Despliegue-Riesgo-Crediticio.

Datos: South German Credit (UCI)

DOI https://doi.org/10.24432/C5QG88.

Como verificación adicional de reproducibilidad, se clonó el repositorio en un directorio independiente y se ejecutó nuevamente la aplicación mediante Docker Compose. Ambos servicios iniciaron correctamente y alcanzaron el estado *healthy*. Las pruebas confirmaron la reproducción de las 200 predicciones históricas y el rechazo de entradas inválidas, sin necesidad de modificar el código ni los componentes del proyecto.

Los resultados quedaron documentados en `evidence/auditoria_reproducibilidad.txt` y en la captura 03 del repositorio.